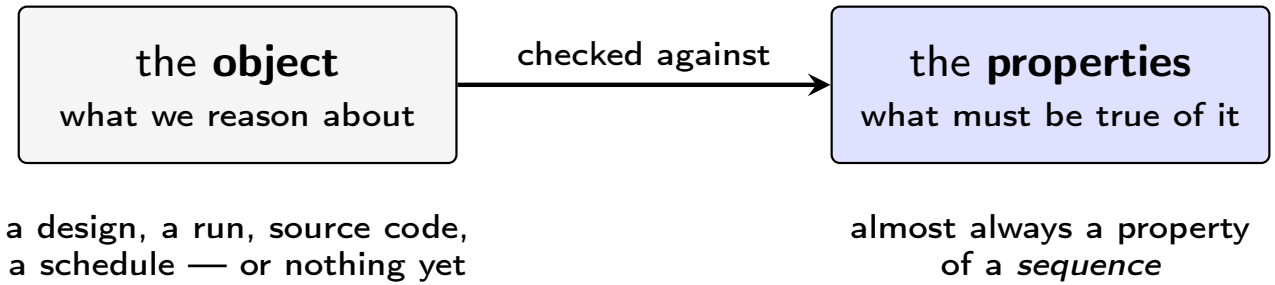
Formal Methods at JPL

Klaus Havelund
Jet Propulsion Laboratory
California Institute of Technology

JPL — September 2026

What this talk is about

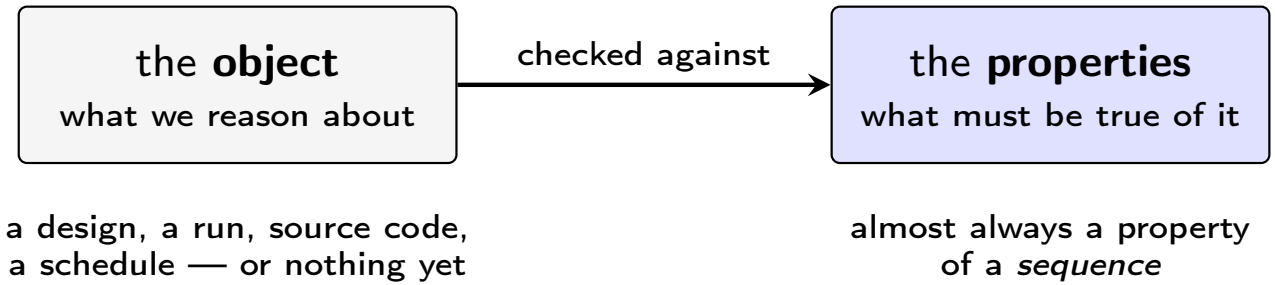
To check anything, you need **two artifacts**:



Both must be written in a notation a machine can read.

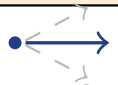
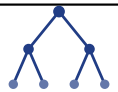
What this talk is about

To check anything, you need **two artifacts**:

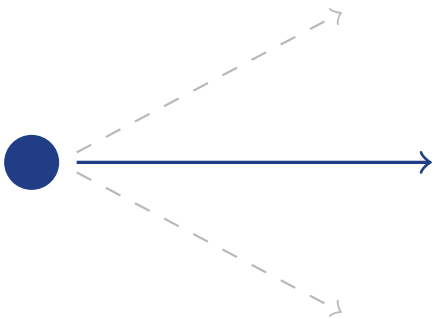


Both must be written in a notation a machine can read.

Five methods

Method	Object	Coverage	Tools
dynamic analysis — the system runs			
 testing	the real system	one run per test	fuzz
 runtime verification	one real execution	that run, exactly	TraceContract, Daut, PyContract, DejaVu
static analysis — the system does not run			
 coding standards	source code	all runs, approximated	JPL checkers
 model checking	a finite-state model	all runs of the *model*	Spin, TLA$^+$, JPF1
 theorem proving	a mathematical model	all runs, unbounded	Lean, Z3, ESBMC, TaskSAT, F$^✓$, EVA, K

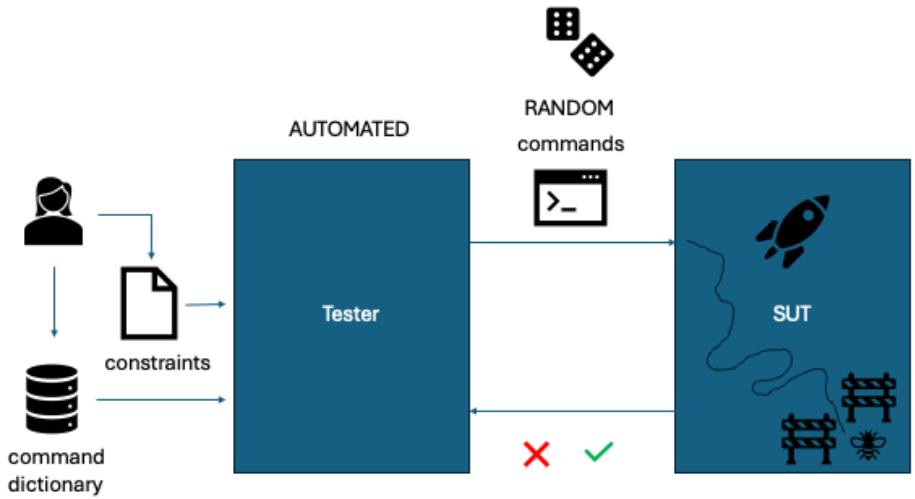
our tools commercial



Testing: the execution does not exist yet

`fuzz`, built on MaTL and Z3

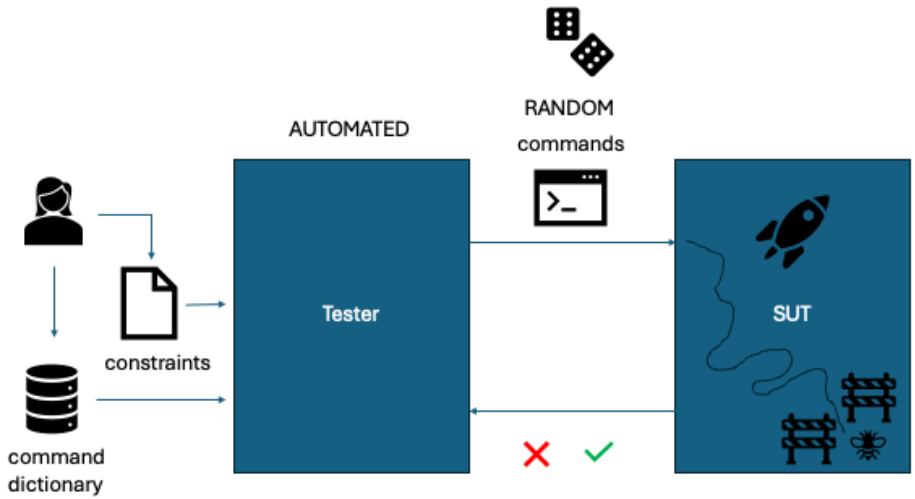
Fuzzing a command-driven system



Spacecraft are operated by **sequences of commands**, tested by hand-written ones that do not reach the corners. Random sequences do, but are rejected as illegal.

So: **constrain the randomness**. The user says what a legal sequence looks like, in temporal logic; we generate random sequences satisfying it.

Fuzzing a command-driven system



Spacecraft are operated by **sequences of commands**, tested by hand-written ones that do not reach the corners. Random sequences do, but are rejected as illegal.

So: **constrain the randomness**. The user says what a legal sequence looks like, in temporal logic; we generate random sequences satisfying it.

From temporal logic to a solver

The specification language is MaTL — temporal logic over events that carry data, with binding. *Every sample collected is eventually downlinked*:

$$\Box(\mathit{COLLECT}(\mathit{file} = f?) \Rightarrow \Diamond\, \mathit{SEND}(\mathit{file} = f))$$

A test is a function from position to command, and the formula becomes a constraint on it. Writing p for the `COLLECT` and q for the matching `SEND`, unfolding over length 3:

$$\begin{aligned} & \neg p(\mathit{test}(0)) \vee q(\mathit{test}(0)) \vee q(\mathit{test}(1)) \vee q(\mathit{test}(2)) \\ \wedge\ & \neg p(\mathit{test}(1)) \vee q(\mathit{test}(1)) \vee q(\mathit{test}(2)) \\ \wedge\ & \neg p(\mathit{test}(2)) \vee q(\mathit{test}(2)) \end{aligned}$$

Hand that to Z3, and a satisfying assignment *is* a legal test:

```
{'name': 'COLLECT', 'number': 1, 'time': 363, 'file': 'img042'}
{'name': 'COLLECT', 'number': 2, 'time': 373, 'file': 'log007'}
{'name': 'SEND',    'number': 3, 'time': 383, 'file': 'img042'}
{'name': 'SEND',    'number': 4, 'time': 394, 'file': 'log007'}
```

Two collections outstanding at once, and *both* downlinked — the binding on f is what makes that a real obligation rather than one `SEND` satisfying everything.

From temporal logic to a solver

The specification language is MaTL — temporal logic over events that carry data, with binding. *Every sample collected is eventually downlinked*:

$$\Box(\mathit{COLLECT}(\mathit{file} = f?) \Rightarrow \Diamond\, \mathit{SEND}(\mathit{file} = f))$$

A test is a function from position to command, and the formula becomes a constraint on it. Writing p for the `COLLECT` and q for the matching `SEND`, unfolding over length 3:

$$\begin{aligned} & \neg p(\mathit{test}(0)) \vee q(\mathit{test}(0)) \vee q(\mathit{test}(1)) \vee q(\mathit{test}(2)) \\ \wedge\ & \neg p(\mathit{test}(1)) \vee q(\mathit{test}(1)) \vee q(\mathit{test}(2)) \\ \wedge\ & \neg p(\mathit{test}(2)) \vee q(\mathit{test}(2)) \end{aligned}$$

Hand that to Z3, and a satisfying assignment *is* a legal test:

```
{'name': 'COLLECT', 'number': 1, 'time': 363, 'file': 'img042'}
{'name': 'COLLECT', 'number': 2, 'time': 373, 'file': 'log007'}
{'name': 'SEND',    'number': 3, 'time': 383, 'file': 'img042'}
{'name': 'SEND',    'number': 4, 'time': 394, 'file': 'log007'}
```

Two collections outstanding at once, and *both* downlinked — the binding on f is what makes that a real obligation rather than one `SEND` satisfying everything.

From temporal logic to a solver

The specification language is MaTL — temporal logic over events that carry data, with binding. *Every sample collected is eventually downlinked*:

$$\Box(\mathit{COLLECT}(\mathit{file} = f?) \Rightarrow \Diamond\, \mathit{SEND}(\mathit{file} = f))$$

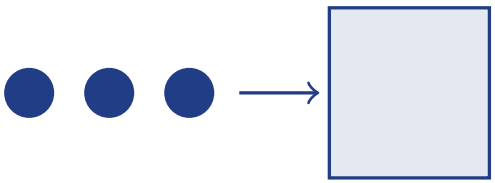
A test is a function from position to command, and the formula becomes a constraint on it. Writing p for the `COLLECT` and q for the matching `SEND`, unfolding over length 3:

$$\begin{aligned} & \neg p(\mathit{test}(0)) \vee q(\mathit{test}(0)) \vee q(\mathit{test}(1)) \vee q(\mathit{test}(2)) \\ \wedge\ & \neg p(\mathit{test}(1)) \vee q(\mathit{test}(1)) \vee q(\mathit{test}(2)) \\ \wedge\ & \neg p(\mathit{test}(2)) \vee q(\mathit{test}(2)) \end{aligned}$$

Hand that to Z3, and a satisfying assignment *is* a legal test:

```
{'name': 'COLLECT', 'number': 1, 'time': 363, 'file': 'img042'}
{'name': 'COLLECT', 'number': 2, 'time': 373, 'file': 'log007'}
{'name': 'SEND',    'number': 3, 'time': 383, 'file': 'img042'}
{'name': 'SEND',    'number': 4, 'time': 394, 'file': 'log007'}
```

Two collections outstanding at once, and *both* downlinked — the binding on f is what makes that a real obligation rather than one `SEND` satisfying everything.

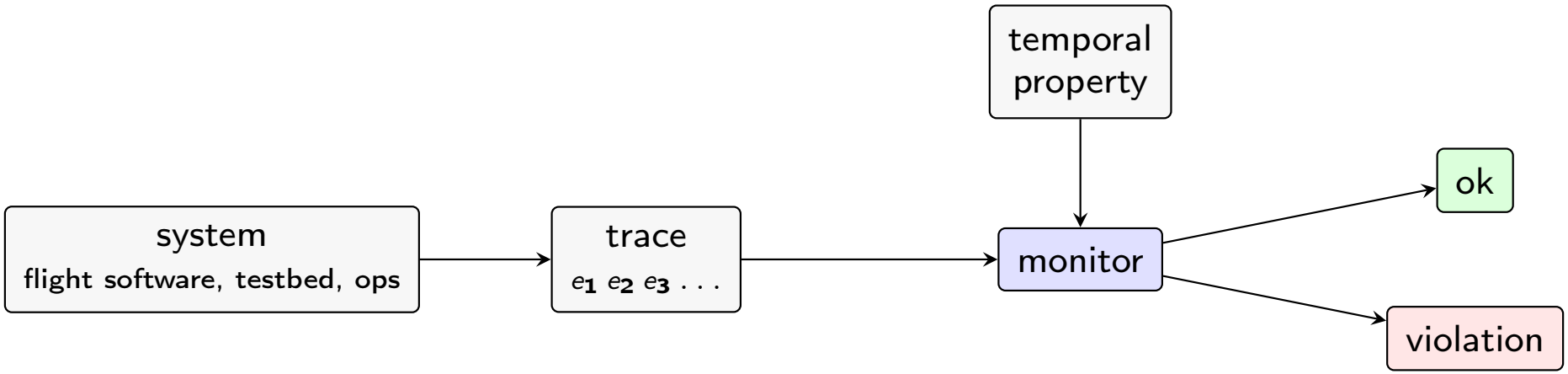


Runtime verification: one execution of the real thing

TraceContract, LogFire, Daut, DejaVu

The run you already have

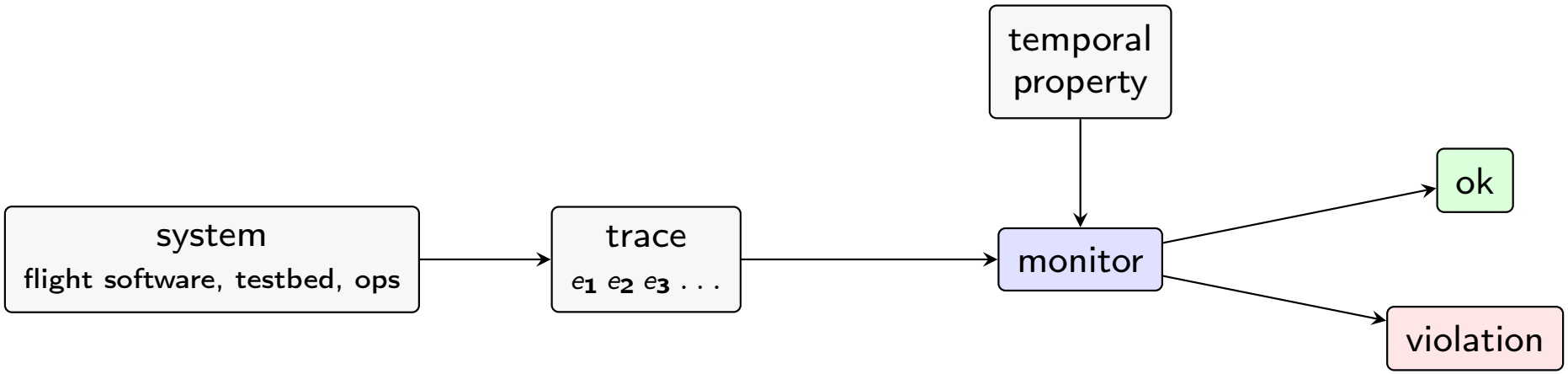
Instrument the code to emit events — or use what it already emits: telemetry, command logs, event reports. Either way you have the thing we want to reason about: a **sequence of events**.



No model to build and justify, and it scales to any system that can be made to talk. The price: it says nothing about the runs you did not observe.

The run you already have

Instrument the code to emit events — or use what it already emits: telemetry, command logs, event reports. Either way you have the thing we want to reason about: a **sequence of events**.



No model to build and justify, and it scales to any system that can be made to talk. The price: it says nothing about the runs you did not observe.

Writing monitors

A monitor is an **internal DSL** — ordinary code in a host language, which supplies the data handling for free. *Every dispatched command must eventually succeed*, in PyContract:

```
class Commands(Monitor):
    @initial
    class Start(AlwaysState):
        def transition(self, event):
            match event:
                case Dispatch(cmd, nr): return self.DoSucceed(cmd, nr)

    @data
    class DoSucceed(HotState):          # an obligation, not yet met
        cmd: str
        nr: int
        def transition(self, event):
            match event:
                case Succeed(self.cmd, self.nr): return ok
                case Fail(_, self.nr):  return error(f'{self.cmd} failed')
```

`HotState` is the point: an obligation still *open* when the trace ends is itself a violation. That is $\Box(p \rightarrow \Diamond q)$ with *nr* bound — and the pattern matching is Python's, not ours.

Writing monitors

A monitor is an **internal DSL** — ordinary code in a host language, which supplies the data handling for free. *Every dispatched command must eventually succeed*, in PyContract:

```
class Commands(Monitor):
    @initial
    class Start(AlwaysState):
        def transition(self, event):
            match event:
                case Dispatch(cmd, nr): return self.DoSucceed(cmd, nr)

    @data
    class DoSucceed(HotState):          # an obligation, not yet met
        cmd: str
        nr: int
        def transition(self, event):
            match event:
                case Succeed(self.cmd, self.nr): return ok
                case Fail(_, self.nr):  return error(f'{self.cmd} failed')
```

`HotState` is the point: an obligation still *open* when the trace ends is itself a violation. That is $\Box(p \rightarrow \Diamond q)$ with *nr* bound — and the pattern matching is Python's, not ours.

DejaVu: an external DSL

DejaVu goes the other way — **its own language**, not a host's. The same obligation, turned into the past: *every command that succeeds was dispatched*.

$$\forall c.\, \forall n.\ \mathit{succeed}(c, n) \rightarrow \diamondsuit\ \mathit{dispatch}(c, n)$$

Fifteen lines of Python become one line of **first-order past LTL** — no state machine, no bookkeeping of which pairs are outstanding. The quantifiers do it.

Under the hood, **BDDs** store the data — which is what makes the monitoring efficient.

DejaVu: an external DSL

DejaVu goes the other way — **its own language**, not a host's. The same obligation, turned into the past: *every command that succeeds was dispatched*.

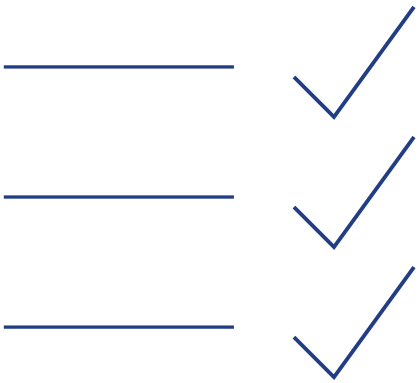
$$\forall c.\, \forall n.\ \mathit{succeed}(c, n) \rightarrow \diamondsuit\ \mathit{dispatch}(c, n)$$

Fifteen lines of Python become one line of **first-order past LTL** — no state machine, no bookkeeping of which pairs are outstanding. The quantifiers do it.

Under the hood, **BDDs** store the data — which is what makes the monitoring efficient.

Five tools, and where they were used

	Host	Formalism	Engine	Applied in
TraceContract	Scala	LTL + state machines	formula rewriting	LADEE
LogFire	Scala	rules, facts in and out	Rete	MSL
Daut	Scala	data automata	indexed state machines	Endurance
PyContract	Python	data automata	indexed state machines	Endurance Europa Clipper
DejaVu	—	first-order past LTL	BDDs over data	—



Coding standards: rules a tool can decide

the cheapest row, and the one with the widest reach

Not everything worth doing is a model checker

Crossing into static analysis — no run at all, just the source.

In 2010–11: JPL's institutional **Java coding standard** and the checker that enforces it, and automated checking of the **C, C++ and Java** standards across the Lab.

Applied and enforced on missions — institutionally, across flight projects.

Cannot check functional correctness — does it do the right thing?

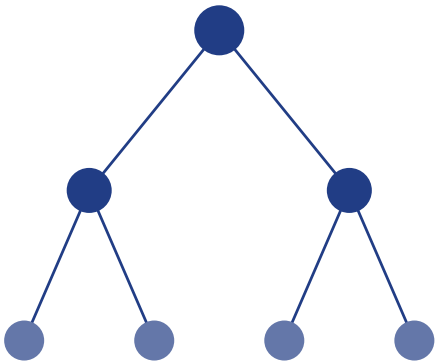
Not everything worth doing is a model checker

Crossing into static analysis — no run at all, just the source.

In 2010–11: JPL's institutional **Java coding standard** and the checker that enforces it, and automated checking of the **C, C++ and Java** standards across the Lab.

Applied and enforced on missions — institutionally, across flight projects.

Cannot check functional correctness — does it do the right thing?



Model checking: all executions of a design

SPIN, and JAVA PATHFINDER 1 (JPF1)

The Deep Space 1 Remote Agent, 1999

First autonomous control of a NASA spacecraft. We analyzed the executive's locking layer: **3000 lines** of multithreaded Lisp, hand-translated to **500 lines** of PROMELA — twelve man-weeks. **Five** flawed fragments, all real.

Then, in flight, 96 million kilometres out: **thrusting did not turn off when requested**. A deadlock — a missing critical section around a conditional wait. **The same defect SPIN had already found**, in code **copied** into another module with the critical section left out. That module was never analyzed.

The Deep Space 1 Remote Agent, 1999

First autonomous control of a NASA spacecraft. We analyzed the executive's locking layer: **3000 lines** of multithreaded Lisp, hand-translated to **500 lines** of PROMELA — twelve man-weeks. **Five** flawed fragments, all real.

Then, in flight, 96 million kilometres out: **thrusting did not turn off when requested**. A deadlock — a missing critical section around a conditional wait. **The same defect SPIN had already found**, in code **copied** into another module with the critical section left out. That module was never analyzed.

A lock-free priority queue, August 2026

Callable from an interrupt handler, so: no mutex, compare-and-swap instead — plus a counter `avail` so a consumer can skip the scan.

```
send(m):     claim FREE slot; write; publish; avail += 1
receive():   if avail == 0: return EMPTY        # peek the counter
             m = getMessage()                   # scan, CAS a READY slot
             if m == empty: return EMPTY
             avail -= 1                         # count down
             return m
```

Two producers, two consumers, two slots; of a slot, only its **state**. SPIN:

```
pan:1: assertion violated (avail>=0) (at depth 11)
  P  publish slot 0, avail++                    avail = 1
  C1 peek: avail > 0, pass     C2 peek: avail > 0, pass
  C1 getMessage -> slot 0, avail--              avail = 0
  P  publish slot 1 -- but avail++ not yet
  C2 getMessage -> slot 1, avail--              avail = -1
```

Both consumers got a message, from *different* slots, so both counted down — but only one `avail++` had landed. The field is `unsigned`: telemetry reads **4 294 967 295**.

The fix: `avail++` *before* publishing. `1992 states, errors: 0`.

A lock-free priority queue, August 2026

Callable from an interrupt handler, so: no mutex, compare-and-swap instead — plus a counter `avail` so a consumer can skip the scan.

```
send(m):     claim FREE slot; write; publish; avail += 1
receive():   if avail == 0: return EMPTY        # peek the counter
             m = getMessage()                   # scan, CAS a READY slot
             if m == empty: return EMPTY
             avail -= 1                         # count down
             return m
```

Two producers, two consumers, two slots; of a slot, only its **state**. SPIN:

```
pan:1: assertion violated (avail>=0) (at depth 11)
  P  publish slot 0, avail++                          avail = 1
  C1 peek: avail > 0, pass    C2 peek: avail > 0, pass
  C1 getMessage -> slot 0, avail--                    avail = 0
  P  publish slot 1 -- but avail++ not yet
  C2 getMessage -> slot 1, avail--                    avail = -1
```

Both consumers got a message, from *different* slots, so both counted down — but only one `avail++` had landed. The field is `unsigned`: telemetry reads **4 294 967 295**.

The fix: `avail++` *before* publishing. `1992 states, errors: 0`.

A lock-free priority queue, August 2026

Callable from an interrupt handler, so: no mutex, compare-and-swap instead — plus a counter `avail` so a consumer can skip the scan.

```
send(m):     claim FREE slot; write; publish; avail += 1
receive():   if avail == 0: return EMPTY        # peek the counter
             m = getMessage()                   # scan, CAS a READY slot
             if m == empty: return EMPTY
             avail -= 1                         # count down
             return m
```

Two producers, two consumers, two slots; of a slot, only its **state**. SPIN:

```
pan:1: assertion violated (avail>=0) (at depth 11)
  P  publish slot 0, avail++                          avail = 1
  C1 peek: avail > 0, pass    C2 peek: avail > 0, pass
  C1 getMessage -> slot 0, avail--                    avail = 0
  P  publish slot 1 -- but avail++ not yet
  C2 getMessage -> slot 1, avail--                    avail = -1
```

Both consumers got a message, from *different* slots, so both counted down — but only one `avail++` had landed. The field is `unsigned`: telemetry reads **4 294 967 295**.

The fix: `avail++` *before* publishing. `1992 states, errors: 0`.



Theorem proving: all executions, no bound

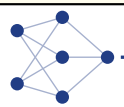
F$^{\checkmark}$ using Lean 4, TaskSAT using Z3,

ESBMC-Python-CPP using ESBMC

F✓: proving an F′ application

NEW **F✓** — a *new* tool of ours, using LEAN 4. The hysteresis logic of `HeaterController`, in flight C++:

```
if (m_controlMode != ControlMode::AUTOMATIC) return;
HeaterState newState = m_heaterState;
if      (avgTemp < (m_targetTemp - m_hysteresis)) newState = ON;
else if (avgTemp > (m_targetTemp + m_hysteresis)) newState = OFF;
if (newState != m_heaterState) commandHeater(newState);
```

and the LEAN model —  **LLM-generated** — tracking it line for line:

```
def avgTempIn_handler (s : ControllerState) (d : AvgTempData) :=
  if s.controlMode != ControlMode.AUTOMATIC then s
  else let newState :=
    if d.avgTemp < s.targetTemp - s.hysteresis then ON
    else if d.avgTemp > s.targetTemp + s.hysteresis then OFF
    else s.heaterState
  if newState != s.heaterState then commandHeater s newState else s
```

F✓: what the proof attempt found

```
theorem cold_reading_turns_heater_on (s) (d)
    (ha    : s.controlMode = ControlMode.AUTOMATIC)
    (hcold : d.avgTemp < s.targetTemp - s.hysteresis) :
    (avgTempIn_handler s d).heaterState = HeaterState.ON
```

Too cold ⇒ heater ON, for *every* state and *every* reading.

Arbitrary operator behaviour

```
theorem executeTrace_preserves_invariant
    (sys) (trace : List SystemEvent)
    (hInv : SystemInvariant sys) :
    SystemInvariant (executeTrace sys trace)
```

No sequence of commands and ticks — of *any* length — can break the invariant. SPIN would have had to bound that list.

F✓: what the proof attempt found

```
theorem cold_reading_turns_heater_on (s) (d)
    (ha    : s.controlMode = ControlMode.AUTOMATIC)
    (hcold : d.avgTemp < s.targetTemp - s.hysteresis) :
    (avgTempIn_handler s d).heaterState = HeaterState.ON
```

Too cold ⇒ heater ON, for *every* state and *every* reading.

Arbitrary operator behaviour

```
theorem executeTrace_preserves_invariant
    (sys) (trace : List SystemEvent)
    (hInv : SystemInvariant sys) :
    SystemInvariant (executeTrace sys trace)
```

No sequence of commands and ticks — of *any* length — can break the invariant. SPIN would have had to bound that list.

TaskSAT: proof without a proof script

NEW a *new* tool for **planning and scheduling**, inspired by MEXEC. A **task net** declares timelines — states, resources, rates — and tasks with windows, preconditions and impacts:

```
battery : rate [10.0, 100.0] bounds [0.0, 100.0] = 60.0;

task drive {
  startrange [100, 120];  durationrange [30, 40];
  pre     { battery in [60.0, 100.0]; }
  impacts { maint { battery +~ −1.5; }  post { location = target; } }
}

constraints { always not (active(charge) and active(drive)); }
properties  { eventually (location = target and data ≥ 30); }
```

Two problems, one encoding

schedule: $\exists\, init.\ \exists\, sched.\ C(init, sched)$ verify: $\forall\, init.\ \forall\, sched.\ C \rightarrow \varphi$

Flip one quantifier and the planner becomes a **prover** — discharged symbolically, for *every* initial battery level and start time, not a state graph.

TaskSAT: proof without a proof script

NEW a *new* tool for **planning and scheduling**, inspired by MEXEC. A **task net** declares timelines — states, resources, rates — and tasks with windows, preconditions and impacts:

```
battery : rate [10.0, 100.0] bounds [0.0, 100.0] = 60.0;

task drive {
  startrange [100, 120];  durationrange [30, 40];
  pre      { battery in [60.0, 100.0]; }
  impacts { maint { battery +~ −1.5; }  post { location = target; } }
}

constraints { always not (active(charge) and active(drive)); }
properties  { eventually (location = target and data ≥ 30); }
```

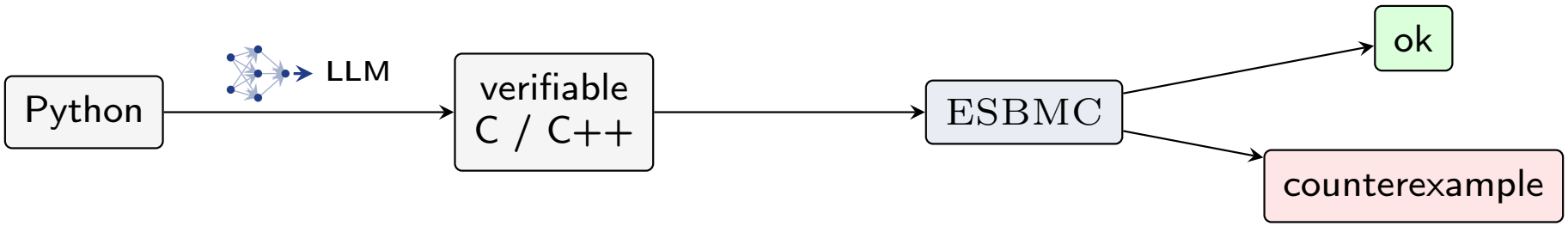
Two problems, one encoding

schedule: $\exists\, init.\ \exists\, sched.\ C(init, sched)$ verify: $\forall\, init.\ \forall\, sched.\ C \rightarrow \varphi$

Flip one quantifier and the planner becomes a **prover** — discharged symbolically, for *every* initial battery level and start time, not a state graph.

ESBMC-Python-CPP: verifying Python

NEW Why Python: we are building a **Python model of an autonomous system** in the **PAAS** project. And Python has no model checker of the kind Spin is — so **translate it into something that does**, and let ESBMC decide.

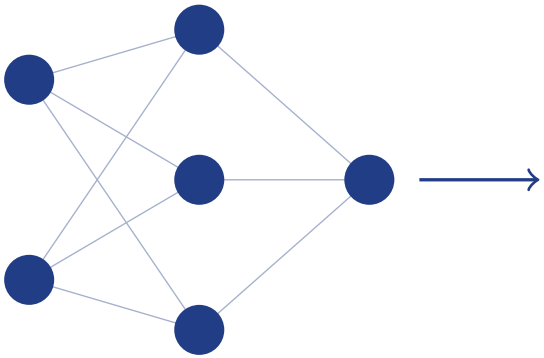


Three routes to the C: **Shedskin** for what converts statically, an **LLM** for what does not, and **dynamic tracing** for what neither can reach.

Driving it: EVA, the **Enhanced Verification Agent** — which also runs the static analysers and the runtime checks, and iterates.

The LLM does the translating. ESBMC does the deciding.

Largely developed by GenAI and the University of Manchester.



LLMs: writing the model was always the expensive part

Who wrote the queue model?

Twelve man-weeks for the Remote Agent model. That cost — not the checking — is why formal methods stayed a specialist activity.

The lock-free queue in Part 4 went differently. An LLM reading the pull request **flagged** the counter as suspicious; asked to demonstrate it, it **wrote the** PROMELA **model**. SPIN then did what SPIN does.

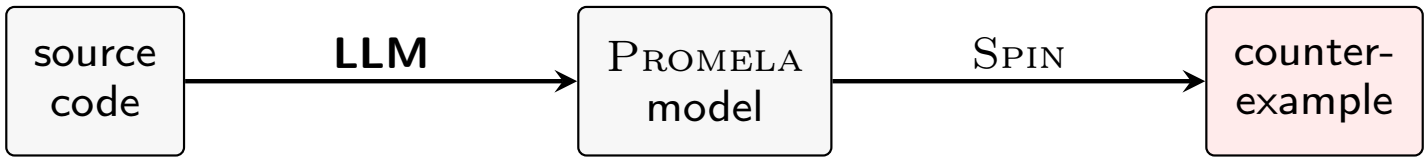
The LLM proposes. The model checker disposes.

An LLM asserting that code is correct is worth little. An LLM producing an artifact that a **sound tool** then checks is worth a great deal — and if the model is wrong, the counterexample is concrete enough to see that it is wrong.

Who wrote the queue model?

Twelve man-weeks for the Remote Agent model. That cost — not the checking — is why formal methods stayed a specialist activity.

The lock-free queue in Part 4 went differently. An LLM reading the pull request **flagged** the counter as suspicious; asked to demonstrate it, it **wrote the** PROMELA **model**. SPIN then did what SPIN does.



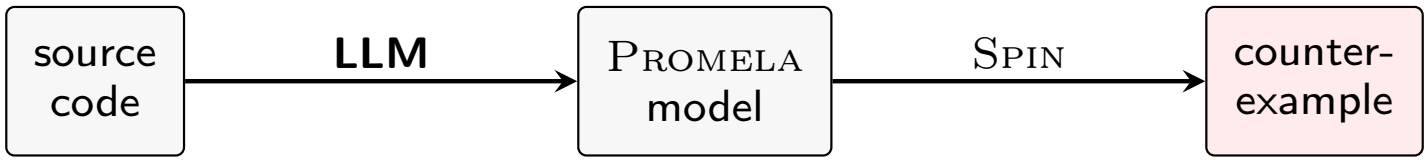
The LLM proposes. The model checker disposes.

An LLM asserting that code is correct is worth little. An LLM producing an artifact that a **sound tool** then checks is worth a great deal — and if the model is wrong, the counterexample is concrete enough to see that it is wrong.

Who wrote the queue model?

Twelve man-weeks for the Remote Agent model. That cost — not the checking — is why formal methods stayed a specialist activity.

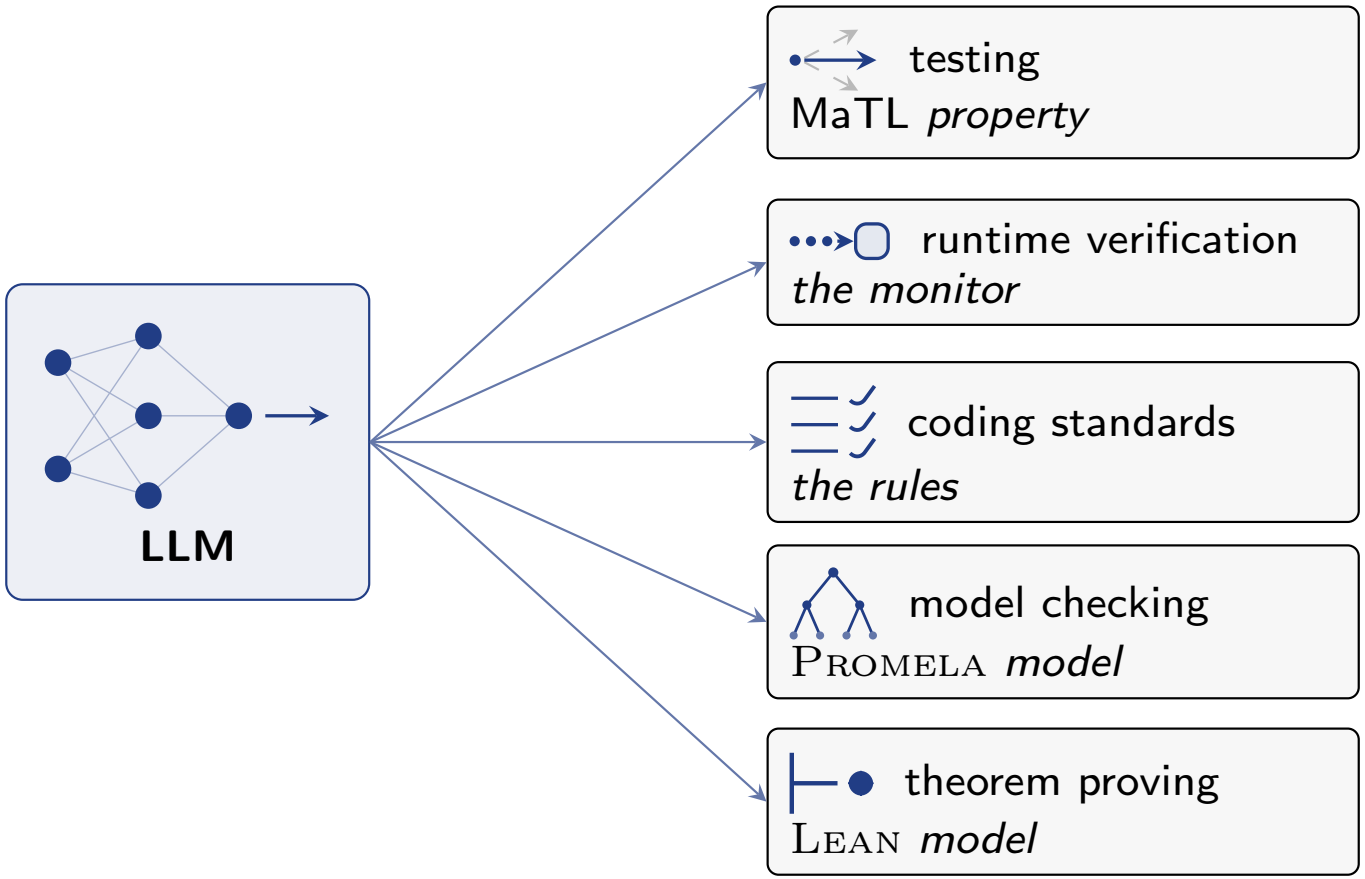
The lock-free queue in Part 4 went differently. An LLM reading the pull request **flagged** the counter as suspicious; asked to demonstrate it, it **wrote the** PROMELA **model**. SPIN then did what SPIN does.



The LLM proposes. The model checker disposes.

An LLM asserting that code is correct is worth little. An LLM producing an artifact that a **sound tool** then checks is worth a great deal — and if the model is wrong, the counterexample is concrete enough to see that it is wrong.

The LLM writes the object — for every one of them



The LLM proposes the **object**. The sound tool still disposes.

Questions?